1. What does DOM stand for in client-side web development?

(A) Document Object Model
(B) Data Object Mechanism
(C) Document Oriented Middleware
(D) Digital Option Model

2. The DOM represents an HTML document in memory as a hierarchical tree of:

(A) Nodes and Objects
(B) Relational Database Tables
(C) Linear Text Buffers
(D) Binary Bitmaps

3. Which global object in browser JavaScript serves as the entry point to the web page's DOM tree?

(A) document
(B) window.screen
(C) navigator
(D) console

4. Which DOM method selects an element based on its unique HTML 'id' attribute?

(A) document.getElementById('id')
(B) document.getElementByClass('id')
(C) document.selectId('id')
(D) document.getId('id')

5. Which modern DOM method returns the FIRST element within the document that matches a specified CSS selector?

(A) document.querySelector('selector')
(B) document.querySelectorAll('selector')
(C) document.getElementByCss('selector')
(D) document.find('selector')

6. Which DOM method returns a static NodeList of ALL elements matching a specified CSS selector?

(A) document.querySelectorAll('selector')
(B) document.querySelector('selector')
(C) document.getElementsByTagName('selector')
(D) document.selectAll('selector')

7. Which property is used to get or set the raw HTML markup contained within an element?

(A) innerHTML
(B) innerText
(C) textContent
(D) outerHTML

8. Which property retrieves or sets the plain rendered text content of an element without HTML tags, respecting CSS styling?

(A) innerText
(B) innerHTML
(C) nodeValue
(D) outerText

9. Which property retrieves or sets all plain text content within a node and its descendants without triggering layout recalculations?

(A) textContent
(B) innerText
(C) innerHTML
(D) value

10. Which DOM method creates a new HTML element node specified by tag name in memory?

(A) document.createElement('tagName')
(B) document.newElement('tagName')
(C) document.buildNode('tagName')
(D) new HTMLElement('tagName')

11. Which DOM method creates a new plain text node in memory?

(A) document.createTextNode('text')
(B) document.createText('text')
(C) new TextNode('text')
(D) document.createString('text')

12. Which DOM method appends a node as the last child of a specified parent element?

(A) parentElement.appendChild(newChild)
(B) parentElement.insert(newChild)
(C) parentElement.attach(newChild)
(D) parentElement.addLast(newChild)

13. Which modern DOM method inserts a node or string before the first child of an element?

(A) element.prepend(newChild)
(B) element.appendChild(newChild)
(C) element.before(newChild)
(D) element.addFirst(newChild)

14. Which modern DOM method allows appending multiple nodes or DOMString objects at once to a parent element?

(A) element.append(...nodes)
(B) element.appendChild(...nodes)
(C) element.push(...nodes)
(D) element.concat(...nodes)

15. Which DOM method removes a specified child node from a parent element?

(A) parentElement.removeChild(childNode)
(B) parentElement.deleteChild(childNode)
(C) childNode.drop()
(D) parentElement.detach(childNode)

16. Which modern DOM method allows an element to remove itself directly from the DOM tree?

(A) element.remove()
(B) element.delete()
(C) element.detach()
(D) element.destroy()

17. Which DOM property provides a live list of an element's CSS class names with helper methods (add, remove, toggle, contains)?

(A) element.classList
(B) element.className
(C) element.classes
(D) element.styles

18. Which method of classList adds a CSS class to an element if it does not already exist?

(A) element.classList.add('className')
(B) element.classList.append('className')
(C) element.classList.push('className')
(D) element.classList.set('className')

19. Which method of classList adds a class if absent, or removes it if already present?

(A) element.classList.toggle('className')
(B) element.classList.switch('className')
(C) element.classList.flip('className')
(D) element.classList.alter('className')

20. Which method of classList checks whether an element currently has a specific CSS class, returning a boolean?

(A) element.classList.contains('className')
(B) element.classList.has('className')
(C) element.classList.includes('className')
(D) element.classList.is('className')

21. Which property on DOM elements allows getting or setting inline CSS styles directly?

(A) element.style
(B) element.css
(C) element.inlineStyle
(D) element.stylesheet

22. How is a multi-word CSS property like 'background-color' accessed via element.style in JavaScript?

(A) element.style.backgroundColor (camelCase)
(B) element.style['background-color']
(C) Both A and B are valid
(D) element.style.background_color

23. Which method attaches an event handler function to an element without overwriting existing handlers?

(A) element.addEventListener(type, listener)
(B) element.attachEvent(type, listener)
(C) element.on(type, listener)
(D) element.bindEvent(type, listener)

24. Which method removes an event listener previously registered with addEventListener?

(A) element.removeEventListener(type, listener)
(B) element.detachEvent(type, listener)
(C) element.off(type, listener)
(D) element.unbind(type, listener)

25. Which event fires when a user clicks on an element with a mouse or pointing device?

(A) 'click'
(B) 'mouseclick'
(C) 'press'
(D) 'tap'

26. Which event fires when the user double-clicks an element?

(A) 'dblclick'
(B) 'doubleclick'
(C) '2click'
(D) 'press2'

27. Which keyboard event fires when a key on the keyboard is initially pushed down?

(A) 'keydown'
(B) 'keypress'
(C) 'keyup'
(D) 'keyhit'

28. Which keyboard event fires when the user releases a pressed key?

(A) 'keyup'
(B) 'keydown'
(C) 'keyrelease'
(D) 'keyend'

29. Which event fires when an input element loses focus (e.g., user tabs or clicks away)?

(A) 'blur'
(B) 'focusout'
(C) 'defocus'
(D) 'leave'

30. Which event fires when an input element gains focus?

(A) 'focus'
(B) 'focusin'
(C) 'active'
(D) 'enter'

31. Which event fires synchronously whenever the value of an <input>, <select>, or <textarea> element is altered by the user?

(A) 'input'
(B) 'change'
(C) 'modify'
(D) 'update'

32. Which event fires when a form element commits a value change (e.g., losing focus after typing, or changing a dropdown selection)?

(A) 'change'
(B) 'input'
(C) 'select'
(D) 'commit'

33. Which event fires when a <form> is submitted by clicking a submit button or pressing Enter?

(A) 'submit'
(B) 'post'
(C) 'send'
(D) 'dispatch'

34. Which method on the Event object cancels the default browser action associated with the event (e.g., preventing form submission or link navigation)?

(A) event.preventDefault()
(B) event.stopPropagation()
(C) event.stopImmediatePropagation()
(D) event.cancel()

35. Which method on the Event object prevents the event from propagating further up or down the DOM tree?

(A) event.stopPropagation()
(B) event.preventDefault()
(C) event.halt()
(D) event.freeze()

36. Which Event property refers to the actual innermost DOM element that originated or triggered the event?

(A) event.target
(B) event.currentTarget
(C) event.srcElement
(D) event.initiator

37. Which Event property refers to the DOM element to which the event listener is currently attached?

(A) event.currentTarget
(B) event.target
(C) event.parentElement
(D) event.boundElement

38. Which event fires when the initial HTML document has been completely parsed without waiting for stylesheets and images to load?

(A) 'DOMContentLoaded'
(B) 'load'
(C) 'ready'
(D) 'mount'

39. Which event on the window object fires when the entire webpage, including all images, stylesheets, and subframes, has fully loaded?

(A) 'load'
(B) 'DOMContentLoaded'
(C) 'complete'
(D) 'finish'

40. Which event fires immediately before the user leaves or closes the webpage, allowing prompt confirmation?

(A) 'beforeunload'
(B) 'unload'
(C) 'pagehide'
(D) 'exit'

41. What are the three distinct phases of DOM Event Flow defined by the W3C specification?

(A) 1. Capturing Phase (Trickling down); 2. Target Phase; 3. Bubbling Phase (Bubbling up)
(B) 1. Init; 2. Dispatch; 3. Destroy
(C) 1. Loading; 2. Active; 3. Idle
(D) 1. Parent phase; 2. Sibling phase; 3. Child phase

42. By default, when you attach an event listener via `element.addEventListener('click', handler)`, during which phase does it fire?

(A) Bubbling Phase (default third argument is false)
(B) Capturing Phase
(C) Target Phase only
(D) Both Capturing and Bubbling

43. How do you configure an event listener to trigger during the Capturing (Trickling) Phase instead of Bubbling?

(A) Pass `true` (or `{ capture: true }`) as the third argument to addEventListener()
(B) Use addCaptureEventListener()
(C) Call event.enableCapture()
(D) Set element.style.capture = true

44. What is Event Delegation in JavaScript DOM programming?

(A) Attaching a single event listener to a common ancestor element to handle events for all existing and dynamically added descendant elements via event bubbling
(B) Delegating event processing to a Web Worker
(C) Passing events to external iframe windows
(D) Binding events to custom objects

45. Why is Event Delegation considered an essential performance pattern when handling large lists (e.g., 10,000 table rows)?

(A) It replaces thousands of individual memory-heavy event listeners with a single listener, drastically lowering memory usage and setup time
(B) It compiles events to C++ machine code
(C) It runs events on the GPU
(D) It prevents garbage collection

46. What is the difference between HTMLCollection and NodeList returned by DOM selection methods?

(A) HTMLCollection is always live (automatically updates when DOM changes) and contains only elements; NodeList can be static (querySelectorAll) and contains any node type
(B) NodeList is an array, HTMLCollection is an object
(C) HTMLCollection supports forEach, NodeList does not
(D) NodeList only holds text nodes

47. Why does `document.getElementsByClassName('item')` differ from `document.querySelectorAll('.item')`?

(A) getElementsByClassName returns a LIVE HTMLCollection; querySelectorAll returns a STATIC NodeList snapshot
(B) querySelectorAll is 100 times slower
(C) getElementsByClassName accepts CSS selectors
(D) querySelectorAll does not work on mobile

48. What does `document.createDocumentFragment()` achieve in DOM manipulation performance?

(A) Creates a lightweight virtual container off-DOM; appending child elements to it and appending the fragment once triggers only ONE single browser reflow
(B) Saves DOM trees to IndexedDB
(C) Creates an HTML iframe
(D) Encrypts DOM nodes in memory

49. Which DOM method provides the computed, resolved styling values of an element as rendered on screen by the browser?

(A) window.getComputedStyle(element)
(B) element.style
(C) element.getStyles()
(D) document.getComputedStyles(element)

50. What is the difference between element.getAttribute('href') and element.href?

(A) element.getAttribute('href') returns the exact string written in HTML markup; element.href returns the fully qualified absolute URL resolved by browser
(B) element.href is read-only
(C) getAttribute is deprecated
(D) element.href only works on images

51. What does the DOM method `element.getBoundingClientRect()` return?

(A) A DOMRect object providing the size (width, height) and position (top, right, bottom, left) of an element relative to the viewport
(B) The size relative to the entire document body
(C) The bounding box of text nodes only
(D) The CSS margin coordinates

52. What is Array and Object Destructuring in ES6?

(A) A syntax expression that unpacks values from arrays or properties from objects into distinct distinct variables cleanly
(B) Deleting items from arrays
(C) Converting arrays into JSON
(D) Serializing objects across sockets

53. Given `const [first, , third = 10] = [1, 2];`, what are the values of 'first' and 'third'?

(A) first = 1, third = 10
(B) first = 1, third = 2
(C) first = 1, third = undefined
(D) first = 2, third = 10

54. Given `const user = { name: 'Bob', age: 30 }; const { name: userName, city = 'Paris' } = user;`, what are userName and city?

(A) userName = 'Bob', city = 'Paris'
(B) name = 'Bob', city = undefined
(C) userName = undefined, city = 'Paris'
(D) Throws ReferenceError

55. What is the Rest Parameter (`...args`) syntax in ES6 function definitions?

(A) Collects an indefinite number of remaining arguments passed to a function into a single real Array
(B) Spreads array items into comma-separated arguments
(C) Converts functions to generators
(D) Limits arguments to 5 items

56. How does the Rest parameter differ from the legacy `arguments` object inside functions?

(A) Rest parameters produce a true JavaScript Array with all array methods; arguments is an array-like object lacking map, filter, and forEach
(B) arguments only works in arrow functions
(C) Rest parameters cannot accept objects
(D) There is zero difference

57. What is the Spread Operator (`...iterable`) syntax in ES6 expressions?

(A) Expands an iterable (like an array or string) into individual elements in function calls, array literals, or object literals
(B) Combines arguments into an array
(C) Creates a closure loop
(D) Deletes array indices

58. What is the result of `const merged = { ...objA, ...objB };` if both objects contain a property with the same key 'id'?

(A) The property value from objB overrides that of objA (properties declared later take precedence)
(B) Throws a duplicate property SyntaxError
(C) Both IDs are merged into an array
(D) objA wins and ignores objB

59. What are ES6 Template Literals, and how do they differ from regular strings?

(A) Strings delimited with backticks (``) allowing embedded expressions via `${expression}` and multi-line strings without newline escape characters
(B) Strings that compile to HTML canvas
(C) Strings encrypted with AES
(D) Strings restricted to 256 characters

60. What is the HTML5 Constraint Validation API method `input.checkValidity()`?

(A) Returns true if the input element satisfies all its validation constraints (required, pattern, min/max), or false otherwise
(B) Submits the form immediately
(C) Displays an alert popup
(D) Resets the form fields

61. What does `input.reportValidity()` do beyond `input.checkValidity()`?

(A) Checks validity AND triggers the browser's native error bubble tooltip if constraints are violated
(B) Clears the input field
(C) Sends error logs to server
(D) Disables the submit button

62. Which method customizes the native browser validation error message on a form input?

(A) input.setCustomValidity('Custom message')
(B) input.setError('Custom message')
(C) input.customError = 'message'
(D) input.validationMessage = 'message'

63. What must be passed to `input.setCustomValidity('')` to clear the custom error and mark the field valid?

(A) An empty string ""
(B) null
(C) false
(D) undefined

64. Which DOM mutation observer API monitors changes to the DOM tree (child additions, attribute modifications, text changes)?

(A) MutationObserver
(B) DOMWatcher
(C) EventListener
(D) ResizeObserver

65. Which modern browser API efficiently monitors changes to an element's physical dimensions (width, height)?

(A) ResizeObserver
(B) MutationObserver
(C) IntersectionObserver
(D) WindowObserver

66. Which modern browser API monitors when a target element enters or exits the browser viewport (ideal for lazy-loading images and infinite scroll)?

(A) IntersectionObserver
(B) ScrollObserver
(C) ViewportWatcher
(D) LazyObserver

67. What is the difference between `element.closest('selector')` and `element.querySelector('selector')`?

(A) `closest()` traverses UP the DOM tree through ancestors looking for a match; `querySelector()` searches DOWN through descendants
(B) Both search downwards
(C) `closest()` only searches sibling nodes
(D) `closest()` is an array method

68. What does `element.matches('selector')` return?

(A) true if the element itself would be selected by the specified CSS selector string; otherwise false
(B) All child elements matching the selector
(C) The parent element
(D) A NodeList

69. What is the difference between `event.stopImmediatePropagation()` and `event.stopPropagation()`?

(A) `stopImmediatePropagation()` prevents bubbling AND stops other listeners registered on the EXACT same element from executing; `stopPropagation()` only halts bubbling to ancestors
(B) `stopImmediatePropagation()` cancels default link behavior
(C) `stopPropagation()` halts JavaScript completely
(D) They are identical aliases

70. What are passive event listeners (`{ passive: true }`) in addEventListener?

(A) Informs the browser the listener will never call `event.preventDefault()`, allowing scrolling and touch gestures to run smoothly without waiting for JavaScript execution
(B) Disables keyboard clicks
(C) Runs listener in background Web Worker
(D) Fires listener only once

71. What does the option `{ once: true }` accomplish in addEventListener?

(A) Automatically removes the event listener after it executes exactly one time
(B) Prevents duplicate clicks within 1 second
(C) Executes listener before DOM loads
(D) Makes event immutable

72. What does the method `element.scrollIntoView({ behavior: 'smooth' })` do?

(A) Smoothly scrolls the browser window until the target element is visible in the viewport
(B) Animates element opacity
(C) Zooms the camera into the element
(D) Refreshes the scrollbar position

73. What is the difference between `window.innerWidth` and `window.outerWidth`?

(A) `innerWidth` is the width of the viewport content area (excluding browser chrome and toolbars); `outerWidth` is the full width of the desktop browser window
(B) `outerWidth` excludes scrollbars
(C) They are identical in full screen
(D) `innerWidth` measures document body

74. How do you dispatch a custom synthetic event in modern JavaScript DOM?

(A) `element.dispatchEvent(new CustomEvent('myEvent', { detail: payload }))`
(B) `element.fireEvent('myEvent', payload)`
(C) `element.trigger('myEvent', payload)`
(D) `document.createEvent('myEvent', payload)`

75. What is the Shadow DOM in Web Components?

(A) An encapsulated, isolated DOM tree attached to an element that prevents internal styles and scripts from leaking into or out of the main document
(B) A hidden backup copy of the DOM
(C) A headless browser instance
(D) A canvas 2D rendering buffer

76. What is Layout Thrashing (Forced Synchronous Layout) in DOM programming, and what causes it?

(A) Alternating between writing to the DOM (style changes) and immediately reading geometric layout properties (offsetHeight, clientWidth), forcing the browser to recalculate layout repeatedly in a loop
(B) Executing too many console.log statements
(C) Loading 50 fonts simultaneously
(D) Animating opacity on the GPU

77. How do you eliminate Layout Thrashing in high-frequency animation loops?

(A) Batch all DOM style reads together first, and then batch all DOM style writes together (or use window.requestAnimationFrame)
(B) Set display: none on the body
(C) Use setTimeout with 1000ms delay
(D) Convert DOM elements to canvas pixels

78. Why should animations be orchestrated using `window.requestAnimationFrame()` rather than `setInterval()` or `setTimeout()`?

(A) `requestAnimationFrame()` synchronizes callbacks with the display hardware refresh rate (60/120 Hz) and pauses automatically when browser tabs are in background, saving CPU/battery
(B) It compiles animations into WebGL
(C) It runs synchronously on the main thread
(D) It bypasses CSS styling rules

79. In an Event Delegation architecture, how do you reliably determine which interactive child button was clicked even if the user clicked an interior nested `<span>` or `<i>` icon?

(A) `const button = event.target.closest('button');` (traverses up from clicked icon to find the enclosing button)
(B) `event.target.querySelector('button')`
(C) `event.currentTarget.parentElement`
(D) `event.target.children[0]`

80. What is the security hazard known as DOM-based Cross-Site Scripting (DOM XSS), and which DOM property is most vulnerable to it?

(A) Insecurely injecting unsanitized user-controlled data into execution sinks like `element.innerHTML` or `eval()`, allowing attacker scripts to run
(B) Using textContent on input fields
(C) Selecting nodes with querySelector
(D) Attaching click event listeners

81. Which modern browser API safely converts an untrusted HTML string into an inert DOM tree sanitized of malicious scripts before insertion?

(A) Sanitizer API (`element.setHTML(untrustedString)`)
(B) JSON.parse()
(C) encodeURIComponent()
(D) document.write()

82. What is the exact functional difference between `element.cloneNode(false)` and `element.cloneNode(true)`?

(A) `cloneNode(false)` performs a shallow clone of the element and attributes only; `cloneNode(true)` performs a deep clone including all child nodes and text content
(B) `true` copies event listeners; `false` does not
(C) `true` copies styles; `false` does not
(D) `false` throws an error if children exist

83. Do cloned DOM nodes created via `element.cloneNode(true)` retain event listeners added via `addEventListener()`?

(A) No: event listeners added via addEventListener are NOT copied; only inline event attributes (onclick) are copied
(B) Yes: all event listeners are cloned automatically
(C) Only click listeners are cloned
(D) Only listeners with passive: true are cloned

84. What does `document.adoptNode(externalNode)` do compared to `document.importNode(externalNode, deep)`?

(A) `adoptNode()` moves the original node from an external document (e.g. iframe) into the current document; `importNode()` creates a new copy leaving the original intact
(B) `importNode()` deletes the original
(C) `adoptNode()` only works on scripts
(D) They are identical aliases

85. In modern ES6+, what does Object Property Shorthand and Computed Property Names achieve?

(A) Shorthand `{ x, y }` maps variables to properties; computed `[prefix + id]: val` allows dynamic runtime expressions inside object literals
(B) Freezes object properties
(C) Generates getters and setters automatically
(D) Converts keys to uppercase

86. What is the output of `const key = 'score'; const obj = { [key]: 100 }; console.log(obj.score);`?

(A) 100 (computed property name evaluates variable 'key')
(B) undefined
(C) NaN
(D) ReferenceError

87. How does Destructuring Assignment handle swapping two variables without a temporary third variable?

(A) `[a, b] = [b, a];`
(B) `a = b = a;`
(C) `swap(a, b);`
(D) `a = [b, b = a][0];` only

88. What is the purpose of `event.isTrusted` boolean property on DOM events?

(A) Returns true if the event was generated by a genuine user action (physical click/keypress), and false if generated by scripts (e.g. element.click())
(B) Verifies whether SSL certificate is trusted
(C) Checks if user is logged in
(D) Verifies CORS origin

89. What happens during Event Bubbling when a click event encounters a shadow root boundary?

(A) The event is retargeted so that `event.target` appears as the custom host element itself to listeners outside the shadow DOM, preserving encapsulation
(B) The event stops immediately and dies
(C) The event reloads the webpage
(D) The event converts into a mouseup event

90. What is the difference between `element.offsetWidth` / `element.offsetHeight` and `element.clientWidth` / `element.clientHeight`?

(A) `offsetWidth/Height` includes content, padding, border, and scrollbar; `clientWidth/Height` includes content and padding only (excludes border and scrollbar)
(B) `clientWidth` includes margins
(C) `offsetWidth` only measures text
(D) There is no difference in border-box

91. What does `element.scrollWidth` / `element.scrollHeight` represent?

(A) The entire width and height of an element's total content, including overflowing content that is currently hidden outside the scrollable viewport
(B) The visible rendered viewport size
(C) The size of the scrollbar thumb
(D) The scroll offset coordinates

92. How do you detect whether an element currently has overflowing content requiring a scrollbar?

(A) `element.scrollHeight > element.clientHeight || element.scrollWidth > element.clientWidth`
(B) `element.style.overflow === 'scroll'`
(C) `element.hasScrollbar()`
(D) `element.offsetHeight > window.innerHeight`

93. What does the DOM method `element.insertAdjacentHTML(position, text)` do, and why is it faster than `innerHTML += text`?

(A) Parses text as HTML and inserts nodes at specified position without re-parsing or destroying existing DOM children; `innerHTML +=` destroys and recreates all children
(B) It creates an iframe
(C) It validates HTML syntax with W3C servers
(D) It compiles HTML into SVG

94. What are the four valid position arguments accepted by `insertAdjacentHTML()`?

(A) 'beforebegin', 'afterbegin', 'beforeend', 'afterend'
(B) 'top', 'bottom', 'left', 'right'
(C) 'first', 'last', 'prepend', 'append'
(D) 'inside', 'outside', 'start', 'end'

95. What is the Virtual DOM used by modern frontend libraries (React, Vue), and how does it optimize real DOM updates?

(A) An in-memory lightweight JavaScript tree representation of the UI; diffing algorithms calculate the minimal delta updates required on the slow real DOM
(B) A browser extension that runs on GPU
(C) A canvas simulation of HTML elements
(D) A WebAssembly engine for CSS

96. What is the difference between shallow and deep reactivity in modern frontend state proxies?

(A) Shallow reactivity only tracks top-level property mutations; deep reactivity recursively wraps nested objects in Proxies to track mutations at all levels
(B) Shallow reactivity uses getters, deep uses setters
(C) Deep reactivity compiles to bytecode
(D) There is no difference in JavaScript

97. What is the role of `document.activeElement` in DOM accessibility and focus management?

(A) Returns the DOM element that currently holds keyboard input focus within the active document
(B) Returns the currently hovered element
(C) Returns the element currently being animated
(D) Returns the active script tag

98. What does the HTML5 'inert' boolean attribute achieve when set on a modal background overlay container?

(A) Makes the entire DOM subtree non-interactive, removing it from tab navigation, accessibility trees, and pointer event hits
(B) Hides the subtree via display: none
(C) Freezes JavaScript execution inside the container
(D) Disables CSS styling

99. How do you safely trap keyboard focus inside an active Modal Dialog for accessibility compliance?

(A) Listen for Tab/Shift+Tab keydown events on first and last focusable elements in modal, wrapping focus in a cycle within the dialog
(B) Disable all keyboard events on window
(C) Set tabIndex = -1 on all buttons
(D) Use alert() popups instead of modals

100. What is the purpose of the modern `AbortController` and `AbortSignal` when managing DOM event listeners?

(A) Allows removing dozens of event listeners across multiple elements simultaneously with a single call to `controller.abort()` via `{ signal: controller.signal }`
(B) Cancels CPU memory allocations
(C) Terminates browser worker processes
(D) Stops CSS transitions midway

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain the Document Object Model (DOM) tree architecture, detailing Node hierarchy: Document, Element, Attr, and Text nodes.

Q2. Compare DOM selection methods: getElementById, getElementsByTagName, getElementsByClassName, querySelector, and querySelectorAll.

Q3. Explain the complete W3C DOM Event Flow architecture: Capturing Phase, Target Phase, and Bubbling Phase.

Q4. Explain Event Delegation in full technical detail, illustrating how to implement a high-performance delegated list handler.

Q5. Analyze Browser Reflow (Layout) and Repaint, detailing Layout Thrashing and strategies to optimize DOM performance.

Q6. Explain the HTML5 Constraint Validation API, detailing form element properties, validity states, and custom validation styling.

Q7. Explain the IntersectionObserver API, detailing its configuration options and demonstrating an image lazy-loading script.

Q8. Explain the MutationObserver API, detailing configuration options and demonstrating tracking of dynamic element insertions.

Q9. Explain ES6+ Destructuring Assignment features: default values, alias renaming, nested destructuring, and parameter destructuring.

Q10. Explain the Rest and Spread operators in ES6+, detailing functional differences across parameters, arrays, and objects.

Q11. Explain Optional Chaining (?.) and Nullish Coalescing (??) operators in ES2020, illustrating bug prevention in nested data.

Q12. Describe the Shadow DOM architecture in Web Components, explaining open vs closed modes, slots, and style scoping.

Q13. Explain modern DOM element insertion methods: append(), prepend(), before(), after(), replaceWith(), and insertAdjacentElement().

Q14. Explain the AbortController API and demonstrate how it manages cleanup of event listeners and cancellation of Fetch requests.

Q15. Describe HTML Form serialization techniques: FormData API, URLSearchParams, and submitting JSON payloads.

Q16. Explain Passive Event Listeners ({ passive: true }) and the performance problems they solve in mobile touch and wheel interactions.

Q17. Discuss Keyboard Accessibility: Focus Trapping, roving tabindex, and ARIA modal accessibility patterns.

Q18. Explain JavaScript Custom Events (new CustomEvent), event bubbling, and component communication architectures.

Q19. Detail DOM security hazards: innerHTML vs textContent, DOM-based XSS, and safe DOM manipulation patterns.

Q20. Explain the ResizeObserver API, compare it with window.onresize, and demonstrate responsive component card resizing.

Q21. Write a complete vanilla JavaScript interactive Tabbed Component: rendering tab buttons, managing active classes, and switching panels.

Q22. Write a comprehensive client-side form validation script that validates username, email, and password complexity in real time.

Q23. Implement an Infinite Scroll feature using IntersectionObserver, fetching and appending new items dynamically when reaching bottom.

Q24. Write an accessible, keyboard-navigable Accordion widget in vanilla JavaScript using ARIA states.

Q25. Demonstrate how to implement a Toast Notification system in pure JavaScript that displays auto-dismissing toast alerts.

Q26. Explain the HTML5 <template> element and write code to clone and populate template content into the DOM.

Q27. Implement a custom Drag-and-Drop Reorderable List in vanilla JavaScript using HTML5 Drag and Drop API.

Q28. Explain JavaScript Event Bubbling traps when working with nested elements and how to handle event.target vs event.currentTarget.

Q29. Write a lightweight state-management store in vanilla JavaScript inspired by Redux (Store, Reducer, Dispatch, Subscribe).

Q30. Analyze DOM performance profiling tools in Chrome DevTools: Performance tab, Memory heap snapshots, and Rendering paint flashing.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[A]	**Q21:**	[A]	**Q41:**	[A]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[A]	**Q22:**	[A]	**Q42:**	[A]	**Q62:**	[A]	**Q82:**	[A]
Q3:	[A]	**Q23:**	[A]	**Q43:**	[A]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[A]	**Q24:**	[A]	**Q44:**	[A]	**Q64:**	[A]	**Q84:**	[A]
Q5:	[A]	**Q25:**	[A]	**Q45:**	[A]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[A]	**Q26:**	[A]	**Q46:**	[A]	**Q66:**	[A]	**Q86:**	[A]
Q7:	[A]	**Q27:**	[A]	**Q47:**	[A]	**Q67:**	[A]	**Q87:**	[A]
Q8:	[A]	**Q28:**	[A]	**Q48:**	[A]	**Q68:**	[A]	**Q88:**	[A]
Q9:	[A]	**Q29:**	[A]	**Q49:**	[A]	**Q69:**	[A]	**Q89:**	[A]
Q10:	[A]	**Q30:**	[A]	**Q50:**	[A]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[A]	**Q31:**	[A]	**Q51:**	[A]	**Q71:**	[A]	**Q91:**	[A]
Q12:	[A]	**Q32:**	[A]	**Q52:**	[A]	**Q72:**	[A]	**Q92:**	[A]
Q13:	[A]	**Q33:**	[A]	**Q53:**	[A]	**Q73:**	[A]	**Q93:**	[A]
Q14:	[A]	**Q34:**	[A]	**Q54:**	[A]	**Q74:**	[A]	**Q94:**	[A]
Q15:	[A]	**Q35:**	[A]	**Q55:**	[A]	**Q75:**	[A]	**Q95:**	[A]
Q16:	[A]	**Q36:**	[A]	**Q56:**	[A]	**Q76:**	[A]	**Q96:**	[A]
Q17:	[A]	**Q37:**	[A]	**Q57:**	[A]	**Q77:**	[A]	**Q97:**	[A]
Q18:	[A]	**Q38:**	[A]	**Q58:**	[A]	**Q78:**	[A]	**Q98:**	[A]
Q19:	[A]	**Q39:**	[A]	**Q59:**	[A]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[A]	**Q40:**	[A]	**Q60:**	[A]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

>>Solution Question Explain Event Delete W3C DOM Event Flow illustration Control in Please. Tamil Performance Debug Plus handler.

```
const listItem = deleteBtn.closest('li');
listItem.remove(); // Removes item
console.log(`Deleted item ID: ${listItem.dataset.id}`);
}
});
```

Benefits: Zero memory overhead, instant setup, and seamless support for dynamically inserted DOM elements.

---

invalid.
3. `element.setCustomValidity(message)`: Injects custom error string. Passing non-empty string marks field invalid; passing empty string `""` marks field valid.

The ValidityState Object (`element.validity`):
- `valueMissing`: True if required field is empty.
- `typeMismatch`: True if value does not match syntax (e.g. invalid email or URL).
- `patternMismatch`: True if value violates regex pattern.
- `tooShort` / `tooLong`: Violates minlength/maxlength.
- `rangeUnderflow` / `rangeOverflow`: Outside min/max numeric bounds.
- `stepMismatch`: Does not fit step interval.
- `customError`: True if custom error message was set.
- `valid`: True if ALL constraints are satisfied.

CSS Pseudo-classes: `:valid`, `:invalid`, `:user-valid`, and `:user-invalid` automatically style fields based on constraint states.

MutationObserver provides an asynchronous mechanism to watch for mutations in the DOM tree, replacing legacy deprecated Mutation Events (like DOMNodeInserted):

Implementation Example:

```javascript
const targetNode = document.getElementById('chatContainer');

const observerOptions = {
childList: true, // Monitor addition/removal of child nodes
subtree: true, // Extend monitoring to all descendant subtrees
attributes: true, // Monitor attribute mutations
attributeFilter: ['class', 'data-status'] // Whitelist specific attributes
};

const observer = new MutationObserver((mutationsList, obs) => {
for (const mutation of mutationsList) {
if (mutation.type === 'childList') {
mutation.addedNodes.forEach(node => {
if (node.nodeType === Node.ELEMENT_NODE) {
console.log('New message element appended:', node);
}
});
}
}
});

observer.observe(targetNode, observerOptions);
// To disconnect: observer.disconnect();
```

>>Solution to Question 9: Explain ES6+ Destructuring Assignment features: default values, alias renaming, nested destructuring, and parameter destructuring.

return `<button style="width: ${width}px">${label}</button>`;
}
```

3. Array Destructuring & Skipping Elements:
`const [first, , third, ...rest] = [10, 20, 30, 40, 50]; // first=10, third=30, rest=[40, 50]`

---
```

```
const timeout1 = userTimeout || 3000; // 3000 (BUG: 0 was intentionally set!)
const timeout2 = userTimeout ?? 3000; // 0 (Correct: retains valid 0)
```

>>Solution Question Shadow DOM architecture in Web Components, explain how `attachShadow()` creates an encapsulated DOM tree, and describe the difference between `appendChild()` and `insertAdjacentElement()`.

- `element.replaceWith(...nodes)`: Replaces `element` directly in its parent with provided nodes.

3. Positioned Insertion (`insertAdjacentElement` / `insertAdjacentHTML`):
Inserts element/HTML at one of four exact relative anchor positions: 'beforebegin', 'afterbegin', 'beforeend', 'afterend'.

---

3. Converting FormData to a Clean JSON Object:
```javascript
const formObject = Object.fromEntries(formData.entries());
const jsonString = JSON.stringify(formObject);
await fetch('/api/user', {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: jsonString
});
```

```
const focusables = modal.querySelectorAll('button, [href], input, select, textarea,
[tabindex]:not([tabindex="-1"])');
const first = focusables[0];
const last = focusables[focusables.length - 1];

if (e.shiftKey && document.activeElement === first) {
last.focus(); e.preventDefault();
} else if (!e.shiftKey && document.activeElement === last) {
first.focus(); e.preventDefault();
}
});
```

4. Focus Restoration: Store a reference to the element that triggered the modal (`document.activeElement`) and return focus to it when the modal closes.

>>Solution to Question 16: DOM Manipulation and Event Handling

The Vulnerability Sink (`innerHTML`):
When untrusted user input (e.g., query params, comments) is assigned to `innerHTML`, attackers can inject malicious scripts:
```javascript
// VULNERABLE TO DOM XSS:
const username = new URLSearchParams(window.location.search).get('user');
profileDiv.innerHTML = `<h1>Welcome ${username}</h1>`;
// Attacker passes: ?user=<img src=x onerror=stealCookies()>
```

Defensive Best Practices:
1. Use `textContent` or `innerText` for plain text insertion; browsers treat input purely as strings, neutralizing tags.
2. Create DOM nodes programmatically: `const h1 = document.createElement('h1'); h1.textContent = username;`.
3. Use the Sanitizer API: `profileDiv.setHTML(untrustedMarkup)` cleanses malicious tags.
4. Enforce strict Content Security Policy (CSP) headers.

[Detailed Step-by-Step Answer]:
Here is a complete, accessible tabbed panel implementation:

```javascript
class TabComponent {
constructor(container) {
this.container = container;
this.tabs = container.querySelectorAll('[role="tab"]');
this.panels = container.querySelectorAll('[role="tabpanel"]');
this.init();
}

init() {
this.container.addEventListener('click', (e) => {
const tab = e.target.closest('[role="tab"]');
if (!tab) return;
this.activateTab(tab);
});
}

activateTab(selectedTab) {
this.tabs.forEach(tab => {
tab.classList.remove('active');
tab.setAttribute('aria-selected', 'false');
});
this.panels.forEach(panel => panel.hidden = true);

selectedTab.classList.add('active');
selectedTab.setAttribute('aria-selected', 'true');
const targetPanelId = selectedTab.getAttribute('aria-controls');
document.getElementById(targetPanelId).hidden = false;
}
}
// Usage: new TabComponent(document.getElementById('myTabGroup'));
```

>>Solution to Question 22: a comprehensive client-side form validation script that validates username, email, and password complexity in real time,

```
if (!isLong || !hasNumber || !hasUpper) {
pwdInput.setCustomValidity('Password must be >= 8 chars with 1 uppercase and 1 digit.');
} else {
pwdInput.setCustomValidity(''); // Clear custom error
}
});

form.addEventListener('submit', (e) => {
if (!form.checkValidity()) {
e.preventDefault();
form.reportValidity(); // Display native error balloons
} else {
console.log('Form inputs validated successfully!');
}
});
```

>> Solution to Question 23: Implement an Infinite Scroll feature using IntersectionObserver, fetching and appending new items dynamically when reaching bottom.

```
  observer.observe(sentinel);
  ```

>>Solution Question 25

```
  container.appendChild(toast);

  setTimeout(() => {
  toast.classList.add('fade-out');
  toast.addEventListener('transitionend', () => toast.remove());
  }, duration);
  }
  }
  // Usage: ToastManager.show('File uploaded successfully!', 'success');
  ```
```

---



>>Solution Question 28: Declare a HTML5 <dialog> element with a unique ID and a close button. Write JavaScript to open the dialog when a button is clicked and close it when the close button is clicked. Use the HTML5 Dialog API.
```

```
list.addEventListener('dragstart', (e) => {
draggedItem = e.target;
e.dataTransfer.effectAllowed = 'move';
setTimeout(() => draggedItem.classList.add('dragging'), 0);
});

list.addEventListener('dragend', () => {
draggedItem.classList.remove('dragging');
draggedItem = null;
});

list.addEventListener('dragover', (e) => {
e.preventDefault();
const afterElement = getDragAfterElement(list, e.clientY);
if (afterElement == null) {
list.appendChild(draggedItem);
} else {
list.insertBefore(draggedItem, afterElement);
}
});

function getDragAfterElement(container, y) {
const elements = [...container.querySelectorAll('li:not(.dragging)')];
return elements.reduce((closest, child) => {
const box = child.getBoundingClientRect();
const offset = y - box.top - box.height / 2;
if (offset < 0 && offset > closest.offset) {
return { offset, element: child };
} else return closest;
}, { offset: Number.NEGATIVE_INFINITY }).element;
}
```

>> Solution to Question 28: Explain JavaScript Event Bubbling traps when working with nested elements and how to handle event.target vs event.currentTarget.

`const button = e.target.closest('.btn'); const id = button.dataset.id;`

- Allocation Instrumentation on Timeline pinpoints functions generating high garbage collection pressure.

3. Rendering Panel (Visual Diagnostics):
- Paint Flashing: Highlights repainted screen regions in green.
- Layout Shift Regions: Highlights elements causing sudden visual jumps (Cumulative Layout Shift / CLS) in blue.
